TALLER DE PROGRAMACIÓN WEB II

Control de flujo interactivo: temporizadores y eventos personalizados

Universidad	Universidad de Caldas	**Asignatura**	Programación Web Creativa II
Docente	Andrés Felipe Osorio Quintero	**Perfil docente**	Ingeniero de Software, desarrollador web y marketing digital
Unidad	2. Interactividad avanzada y almacenamiento local		
Subtema	Control de flujo interactivo		
Documento fuente	1-Control_de_Flujo_Interactivo.pdf		
Código base	Codigo HTML CSS JS.docx (lista de tareas)		
Estudiante	**Kevin Ospina y Valentina Muñoz**	**Fecha**	29/10/2026

Indicaciones: Responda las preguntas 1 a 5 con sus propias palabras. Desarrolle los ejercicios 6 a 20 en Visual Studio Code, tomando como punto de partida el código base y siguiendo el orden propuesto, y adjunte las evidencias solicitadas.

Parte I. Teoría (Preguntas 1 a 5)

1. Modelo de ejecución del navegador

Explique con sus propias palabras cómo ejecuta el navegador el código JavaScript de una página, relacionando el hilo principal, la pila de llamadas, las Web APIs, las colas de tareas y de microtareas, y el bucle de eventos. Con base en ese modelo, argumente por qué una promesa ya resuelta se atiende antes que un setTimeout con retardo cero.

Respuesta:
JavaScript es monohilo. El **Hilo Principal** ejecuta el código síncrono en la **Pila de Llamadas (Call Stack)**. Las tareas asíncronas pasan a las **Web APIs**, y al finalizar se envían a sus colas:
• **Cola de Microtareas:** Promesas (`.then()`).
• **Cola de Macrotareas:** `setTimeout` y `setInterval`.

El **Event Loop** revisa la Pila y, cuando está vacía, procesa primero **toda** la Cola de Microtareas antes de atender una Macrotarea. Por ello, una promesa resuelta siempre se ejecuta antes que un `setTimeout(..., 0)`.

2. El retardo de un temporizador

¿Por qué se afirma que el retardo de setTimeout es un tiempo mínimo y no una garantía de puntualidad? Analice al menos dos situaciones que pueden retrasar un temporizador y explique qué consecuencias tienen para una cuenta regresiva que resta un segundo en cada ejecución.

Respuesta:
El retardo indica cuándo pasa el callback a la cola de tareas, no cuándo se ejecuta (esto depende de que la Pila esté libre).

Factores de retraso:
1. **Bloqueo del Hilo Principal:** Tareas síncronas pesadas que ocupan la Pila.
2. **Pestañas inactivas:** Los navegadores reducen la frecuencia de temporizadores en segundo plano.

Consecuencia: Restar un segundo fijo por iteración acumula desfases temporales, haciendo que la cuenta regresiva sea más lenta que el tiempo real.

3. setInterval frente a setTimeout recursivo

Compare setInterval y setTimeout recursivo. Argumente cuál utilizaría en un visor de cuenta regresiva y cuál en la consulta periódica a un servicio remoto, y explique qué precauciones debe tomar con los identificadores para evitar intervalos duplicados y fugas de memoria.

Respuesta:

• `setInterval` ejecuta código a intervalos fijos sin importar si la llamada anterior terminó.
• `setTimeout` recursivo espera a que termine la ejecución actual antes de programar la siguiente.

Uso ideal: `setInterval` para **cuentas regresivas** (ticks simples de UI) y `setTimeout` recursivo para **polling a servicios** (evita solapar peticiones si la red es lenta).

Prevención: Guardar el ID retornado y llamar a `clearInterval()` / `clearTimeout()` antes de iniciar un nuevo temporizador.

4. Eventos personalizados con CustomEvent

Explique para qué sirve CustomEvent y cómo influyen las opciones detail, bubbles, cancelable y composed en su comportamiento. Interprete por qué un evento emitido por un botón con los valores por defecto no llega a los escuchadores de document, y qué implica que dispatchEvent sea síncrono.

Respuesta:

`CustomEvent` permite crear eventos propios para comunicar módulos de forma desacoplada.
• `detail`: Datos adjuntos. | `bubbles`: Propagación hacia arriba en el DOM.
• `cancelable`: Permite `preventDefault()`. | `composed`: Atraviesa Shadow DOM.

Un evento de botón no llega a `document` por defecto porque `bubbles` es `false`. Además, `dispatchEvent` es **síncrono**: ejecuta todos sus escuchadores inmediatamente antes de continuar el código.

5. Interfaz web y procesos ETL

Analice qué aportan los temporizadores y los eventos personalizados a un proceso de datos y qué no pueden hacer. Argumente por qué setInterval no es un planificador de procesos ETL y qué le corresponde al navegador, al servidor de aplicación y a la infraestructura ETL especializada.

Respuesta:

Aportan reactividad visual y control de frecuencia en la UI, pero **no procesan grandes volúmenes de datos** ni garantizan persistencia o tolerancia a fallos.

`setInterval` no es un planificador ETL porque se detiene al cerrar la pestaña y carece de control transaccional.

Responsabilidades:

• **Navegador:** Interfaz, eventos y visualización de estado.
• **Servidor App:** Autenticación y lógica de API.
• **Infraestructura ETL:** Procesamiento pesado, colas de tareas y persistencia.

Parte II. Práctica (Ejercicios 6 a 20)

6. Orden de ejecución

¿En qué orden se muestran en la consola un código síncrono, un setTimeout con retardo 0 y una promesa ya resuelta?

Predicción: 1. "Inicio" (síncrono) → 2. "Fin" (síncrono) → 3. "Promesa resuelta" (Microtarea) → 4. "Timeout 0" (Macrotarea).

Evidencia 1: captura del código en Visual Studio Code

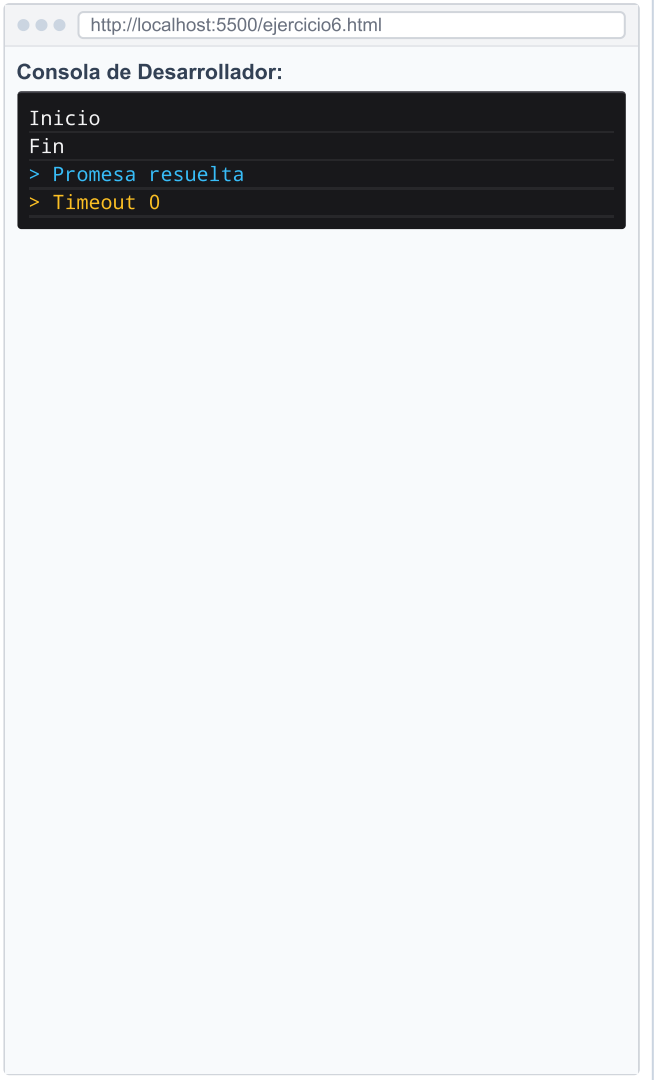
```
// Ejercicio 6 - Orden de Ejecución
console.log("Inicio");

setTimeout(() => {
  console.log("Timeout 0");
}, 0);

Promise.resolve().then(() => {
  console.log("Promesa resuelta");
});

console.log("Fin");
```

Evidencia 2: captura del resultado en el navegador



7. Mensaje temporal

¿Cómo confirmar al usuario que una tarea fue agregada sin que deba cerrar el mensaje manualmente?

Evidencia 1: captura del código en Visual Studio Code

```
let timerId = null;

function mostrarNotificacion(msg) {
  const status = document.querySelector('[role="st
atus"]');
  status.textContent = msg;
  status.style.display = 'block';

  timerId = setTimeout(() => {
    status.style.display = 'none';
  }, 4000);
}
```

Evidencia 2: captura del resultado en el navegador



8. Reinicio del temporizador

¿Qué ocurre si se agregan dos tareas seguidas antes de que venzan los 4 segundos?

Evidencia 1: captura del código en Visual Studio Code

```
let timerCierre = null;

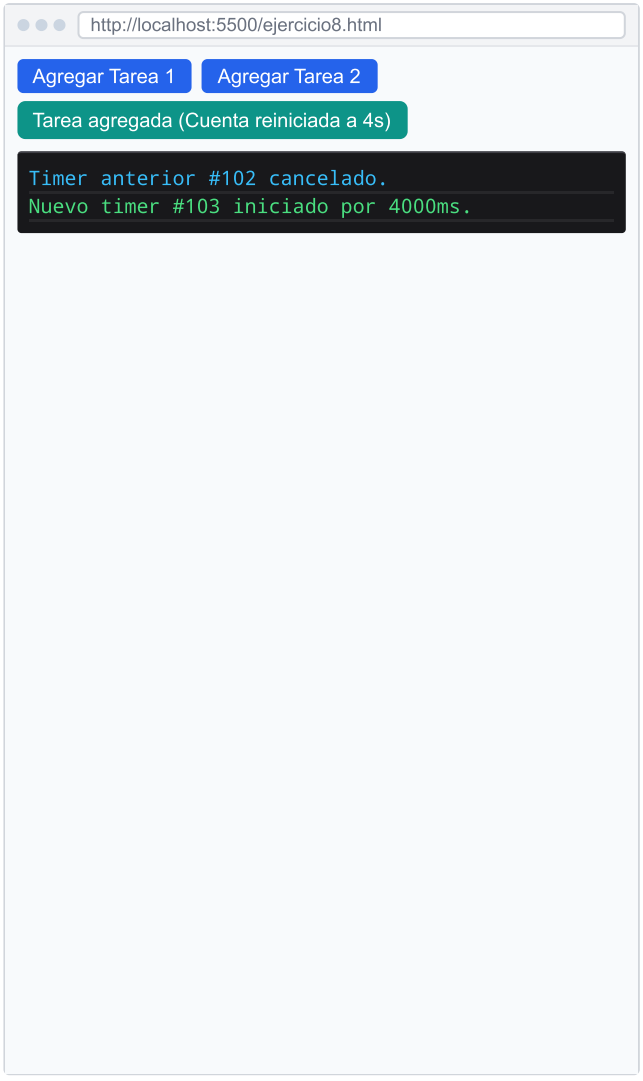
function programarCierre() {
  const status = document.querySelector('[role="st
atus"]');

  // Limpia temporizador anterior si existía
  if (timerCierre) clearTimeout(timerCierre);

  status.textContent = "Tarea agregada";
  status.style.display = "block";

  timerCierre = setTimeout(() => {
    status.style.display = "none";
  }, 4000);
}
```

Evidencia 2: captura del resultado en el navegador



9. Pausa del aviso

¿Por qué un aviso que desaparece solo puede ser un problema de accesibilidad?

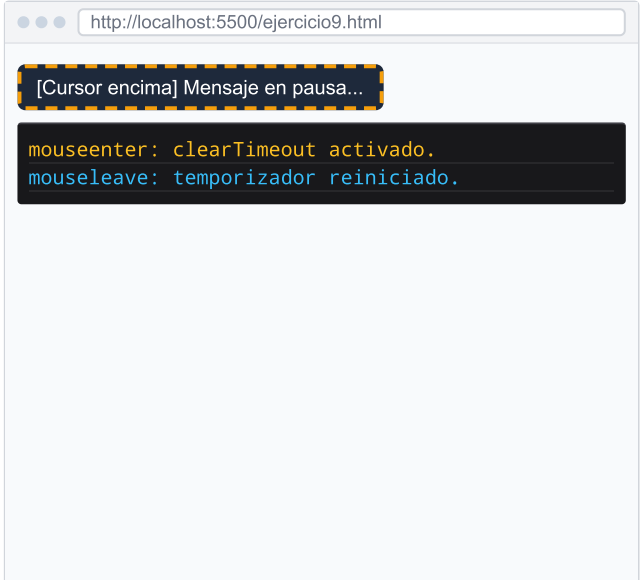
Evidencia 1: captura del código en Visual Studio Code

```
const statusEl = document.querySelector('[role="st
atus"]');

statusEl.addEventListener('mouseenter', () => {
  clearTimeout(timerCierre); // Pausa
});

statusEl.addEventListener('mouseleave', () => {
  programarCierre(); // Reanuda
});
```

Evidencia 2: captura del resultado en el navegador



10. Cuenta regresiva precisa

¿Cómo lograr que una cuenta regresiva no se atrase aunque el intervalo no sea puntual?

Evidencia 1: captura del código en Visual Studio Code

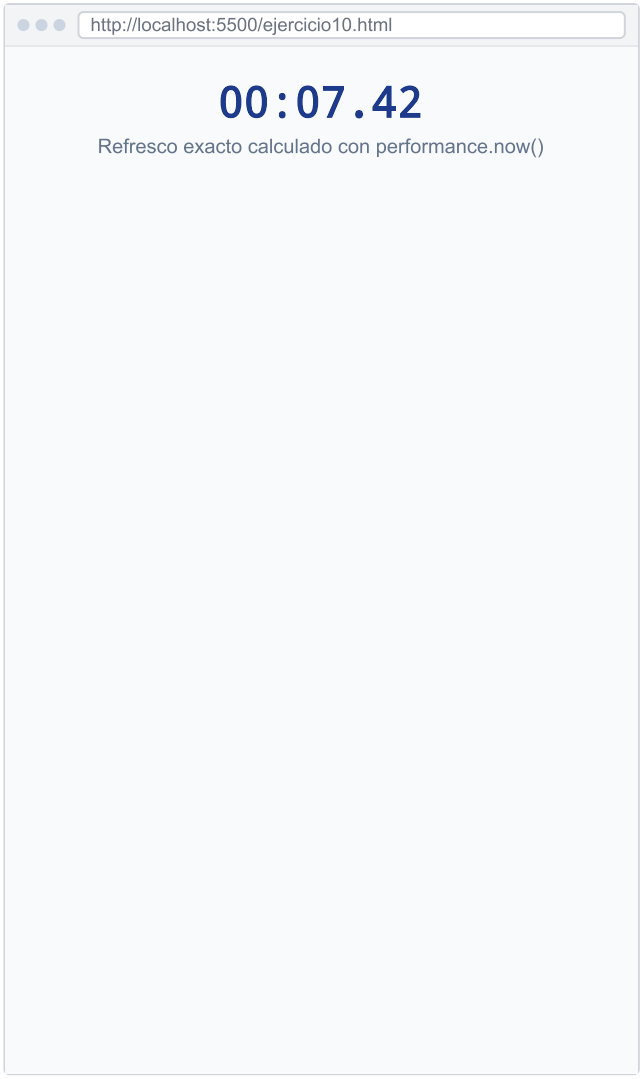
```
const duracion = 10000; // 10 segundos
const inicio = performance.now();

const timer = setInterval(() => {
  const transcurrido = performance.now() - inicio;
  const restante = Math.max(0, duracion - transcur
rido);
  const seg = (restante / 1000).toFixed(2);

  actualizarVisor(`00:${seg}`);

  if (restante <= 0) {
    clearInterval(timer);
    actualizarVisor("¡Tiempo!");
  }
}, 250);
```

Evidencia 2: captura del resultado en el navegador



11. Controles de la cuenta regresiva

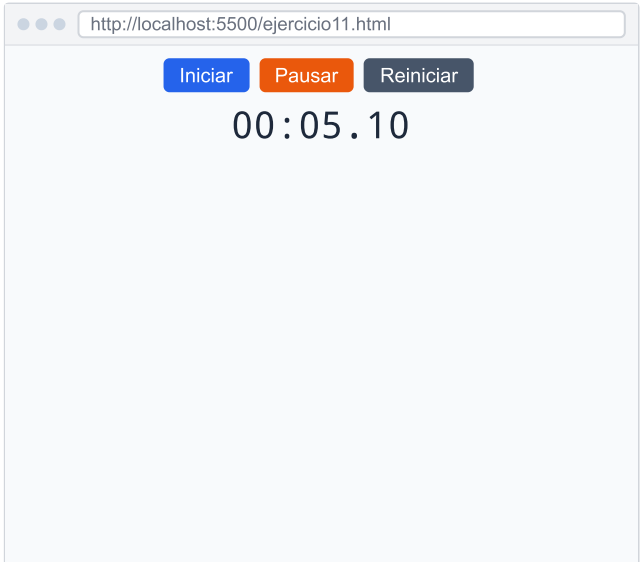
¿Qué sucede si el botón Iniciar se pulsa varias veces seguidas?

Evidencia 1: captura del código en Visual Studio Code

```
let timerId = null;
let tiempoRestante = 10000;

function iniciar() {
  if (timerId !== null) return; // Evita duplicado
s
  const inicio = performance.now();
  timerId = setInterval(() => {
    // lógica de descuento...
  }, 250);
}
function pausar() { clearInterval(timerId); timerI
d = null; }
```

Evidencia 2: captura del resultado en el navegador



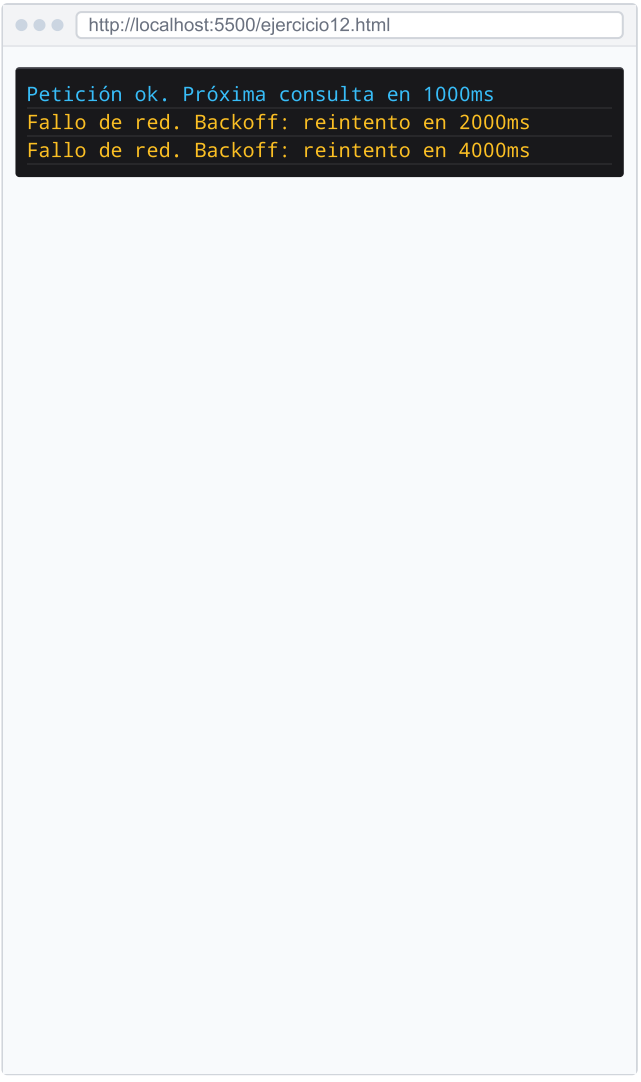
12. Consulta periódica con reintentos

¿Por qué es preferible un setTimeout recursivo a un setInterval para consultar un servicio?

Evidencia 1: captura del código en Visual Studio Code

```
let espera = 1000;
function consultarEstado() {
  fetchEstado().then(res => {
    espera = 1000; // Reset tras éxito
    setTimeout(consultarEstado, espera);
  }).catch(() => {
    espera = Math.min(espera * 2, 8000); // Backof
f
    setTimeout(consultarEstado, espera);
  });
}
```

Evidencia 2: captura del resultado en el navegador



13. Filtro con debounce

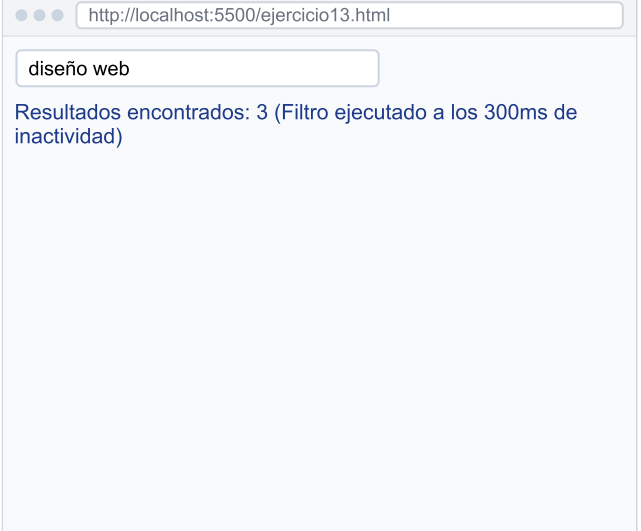
¿Cómo evitar que un filtro se ejecute en cada pulsación de tecla?

Evidencia 1: captura del código en Visual Studio Code

```
let debounceTimer;
const inputSearch = document.querySelector('#searc
h');

inputSearch.addEventListener('input', (e) => {
  clearTimeout(debounceTimer);
  debounceTimer = setTimeout(() => {
    const query = e.target.value;
    filtrarTareas(query);
  }, 300);
});
```

Evidencia 2: captura del resultado en el navegador



14. Primer evento personalizado

¿Cómo anunciar un hecho propio de la aplicación con el mismo mecanismo de los eventos nativos?

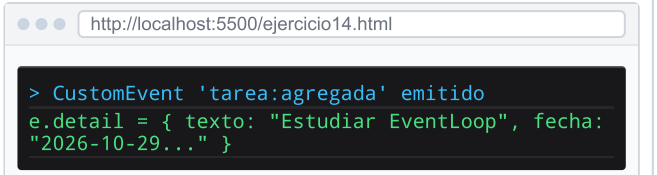
Evidencia 1: captura del código en Visual Studio Code

```
function agregarTarea(texto) {
  const evento = new CustomEvent('tarea:agregada',
{
    bubbles: true,
    detail: { texto, fecha: new Date() }
  });

  btnAgregar.dispatchEvent(evento);
}

document.addEventListener('tarea:agregada', (e) =>
{
  console.log("Evento capturado:", e.detail.text
o);
});
```

Evidencia 2: captura del resultado en el navegador



15. Módulos desacoplados

¿Cómo actualizar un contador sin que el código de la lista llame funciones del contador?

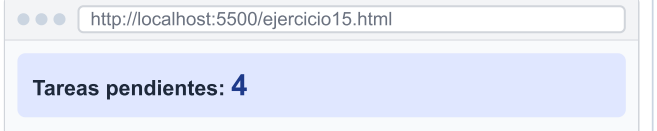
Evidencia 1: captura del código en Visual Studio Code

```
// Módulo Contador Totalmente Desacoplado
let total = 0;
const contadorEl = document.querySelector('#tota
l');

document.addEventListener('tarea:agregada', () =>
{
  total++;
  contadorEl.textContent = total;
});

document.addEventListener('tarea:eliminada', () =>
{
  total = Math.max(0, total - 1);
  contadorEl.textContent = total;
});
```

Evidencia 2: captura del resultado en el navegador



16. Evento cancelable

¿Cómo permitir que otro módulo impida una acción antes de que ocurra?

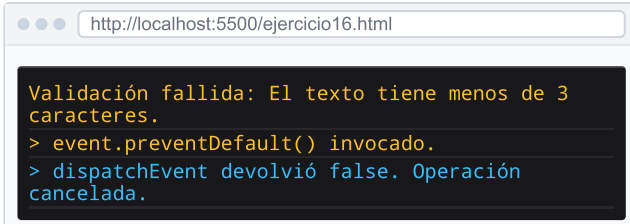
Evidencia 1: captura del código en Visual Studio Code

```
const eventoAntes = new CustomEvent('tarea:antes-agregar', {
  cancelable: true,
  detail: { texto }
});

const permitido = document.dispatchEvent(eventoAntes);

if (permitido) {
  insertarEnLista(texto);
} else {
  console.warn("Inserción cancelada por validador.");
}
```

Evidencia 2: captura del resultado en el navegador



17. Retiro de escuchadores

¿Cómo evitar acciones repetidas y fugas de memoria al dejar de usar un componente?

Evidencia 1: captura del código en Visual Studio Code

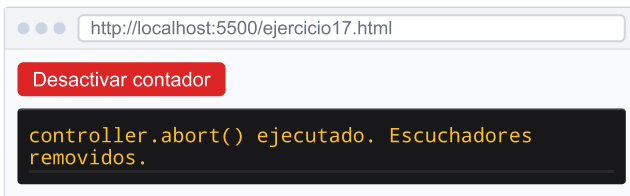
```
const controller = new AbortController();
const { signal } = controller;

document.addEventListener('tarea:agregada', fnListener, { signal });

// Escuchador de ejecución única
document.addEventListener('bienvenida', mostrarIntro, { once: true });

document.querySelector('#btnDesactivar').addEventListener('click', () => {
  controller.abort(); // Remueve todos los listeners asociados
});
```

Evidencia 2: captura del resultado en el navegador



18. Servicio con EventTarget

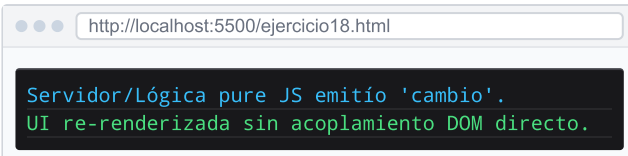
¿Cómo aplicar el modelo de eventos a la lógica de negocio sin depender del DOM?

Evidencia 1: captura del código en Visual Studio Code

```
class GestorTareas extends EventTarget {
  agregar(tarea) {
    this.dispatchEvent(new CustomEvent('cambio', {
detail: tarea }));
  }
}

const gestor = new GestorTareas();
gestor.addEventListener('cambio', (e) => {
  renderizarUI(e.detail);
});
```

Evidencia 2: captura del resultado en el navegador



Evidencia 1: captura del código en Visual Studio Code

```
function asignarLimite(tareaId, limiteSeg) {
  const timerId = setTimeout(() => {
    document.dispatchEvent(new CustomEvent('tarea:
vencida', {
      detail: { tareaId }
    }));
  }, limiteSeg * 1000);
}
```

Evidencia 2: captura del resultado en el navegador



20. Caso integrador: monitor ETL simulado

¿Cómo separar un servicio que conduce un proceso en el tiempo de la interfaz que lo muestra?

Evidencia 1: captura del código en Visual Studio Code

```
class MonitorETL extends EventTarget {
  iniciar(origen) {
    const ev = new CustomEvent('etl:inicio', { can
celable:true, detail:{origen} });
    if(!this.dispatchEvent(ev)) return;
    // Etapas: Extracción -> Transformación -> Car
ga
  }
}
```

Evidencia 2: captura del resultado en el navegador

